

Continuous exposures and g-computation

Malcolm Barrett

Stanford University

Normal regression estimates associations. But we want *causal* estimates: what would happen if *everyone* in the study were exposed to x vs if *no one* was exposed.

G-Computation/G-Formula

- 1 Fit a model for $y \sim x + z$ where z is all covariates
- 2 Create a duplicate of your data set for each level of x
- 3 Set the value of x to a single value for each cloned data set (e.g $x = 1$ for one, $x = 0$ for the other)

G-Computation/G-Formula

- 1 Make predictions using the model on the cloned data sets
- 2 Calculate the estimate you want, e.g. $\text{mean}(x_1) - \text{mean}(x_0)$

Advantages of the parametric G-formula

Often more statistically precise than propensity-based methods

Incredibly flexible

Basis of other important causal models, e.g. causal survival analysis and TMLE

Greek Pantheon data (greek_data)

The name of a Greek god	A prognostic factor	The treatment, a heart transplant	The outcome, death
Rheia	0	0	0
Kronos	0	0	1
Demeter	0	0	0
Hades	0	0	0
Hestia	0	1	0
Poseidon	0	1	0
Hera	0	1	0
Zeus	0	1	1
Artemis	1	0	1
Apollo	1	0	1

+ 10 more rows

1. Fit a model for $y \sim a + 1$

```
1 greek_model <- lm(y ~ a + 1, data = greek_data)
```

2. Create a duplicate of your data set for each level of a

The name of a Greek god	A prognostic factor	The treatment, a heart transplant	The outcome, death
Rheia	0	0	0
Kronos	0	0	1
Demeter	0	0	0
Hades	0	0	0
Hestia	0	1	0
Poseidon	0	1	0
Hera	0	1	0
Zeus	0	1	1
Artemis	1	0	1
Apollo	1	0	1

2. Create a duplicate of your data set for each level of a

The name of a Greek god	A prognostic factor	The treatment, a heart transplant	The outcome, death
Rheia	0	0	0
Kronos	0	0	1
Demeter	0	0	0
Hades	0	0	0
Hestia	0	1	0
Poseidon	0	1	0
Hera	0	1	0
Zeus	0	1	1
Artemis	1	0	1
Apollo	1	0	1

The name of a Greek god	A prognostic factor	The treatment, a heart transplant	The outcome, death
Rheia	0	0	0
Kronos	0	0	1
Demeter	0	0	0
Hades	0	0	0
Hestia	0	1	0
Poseidon	0	1	0
Hera	0	1	0
Zeus	0	1	1
Artemis	1	0	1
Apollo	1	0	1

3. Set the value of **a** to a single value for each cloned data set

The name of a Greek god	A prognostic factor	a	The outcome, death
Rheia	0	0	0
Kronos	0	0	1
Demeter	0	0	0
Hades	0	0	0
Hestia	0	0	0
Poseidon	0	0	0
Hera	0	0	0
Zeus	0	0	1
Artemis	1	0	1
Apollo	1	0	1

The name of a Greek god	A prognostic factor	a	The outcome, death
Rheia	0	1	0
Kronos	0	1	1
Demeter	0	1	0
Hades	0	1	0
Hestia	0	1	0
Poseidon	0	1	0
Hera	0	1	0
Zeus	0	1	1
Artemis	1	1	1
Apollo	1	1	1

3. Set the value of **a** to a single value for each cloned data set

```
1 # set all participants to have a = 0
2 untreated_data <- greek_data |>
3   mutate(a = 0)
4
5 # set all participants to have a = 1
6 treated_data <- greek_data |>
7   mutate(a = 1)
```

4. Make predictions using the model on the cloned data sets

```
1 # predict under the data where everyone is untreated
2 predicted_untreated <- greek_model |>
3   augment(newdata = untreated_data) |>
4   select(untreated = .fitted)
5
6 # predict under the data where everyone is treated
7 predicted_treated <- greek_model |>
8   augment(newdata = treated_data) |>
9   select(treated = .fitted)
10
11 predictions <- bind_cols(
12   predicted_untreated,
13   predicted_treated
14 )
```

5. Calculate the estimate you want

```
1 predictions |>
2   summarise(
3     mean_treated = mean(treated),
4     mean_untreated = mean(untreated),
5     difference = mean_treated - mean_untreated
6   )
```

A tibble: 1 × 3

	mean_treated	mean_untreated	difference
	<dbl>	<dbl>	<dbl>
1	0.5	0.5	0

avg_comparisons() does all five steps

```
1 library(marginaleffects)
2
3 avg_comparisons(greek_model, variables = "a")
```

```
# A tibble: 1 × 4
  estimate std.error conf.low conf.high
  <dbl>    <dbl>    <dbl>    <dbl>
1      0      0.24   -0.47     0.47
```

The standard error comes from the **delta method**, which approximates the variance of a transformation of the estimated coefficients

Targeting estimands with **newdata**

newdata changes whose covariates we average over

```
1  # ATT: average over the treated
2  avg_comparisons(
3    greek_model,
4    variables = "a",
5    newdata = filter(greek_data, a == 1)
6  )
7
8  # ATC: average over the untreated
9  avg_comparisons(
10   greek_model,
11   variables = "a",
12   newdata = filter(greek_data, a == 0)
13 )
```

Same model, three populations

```
# A tibble: 3 × 4
  estimand estimate conf.low conf.high
<chr>      <dbl>    <dbl>    <dbl>
1 ATE          0    -0.47     0.47
2 ATT          0    -0.47     0.47
3 ATC          0    -0.47     0.47
```

Without an exposure-covariate interaction, the three coincide: the model assumes a constant effect across covariate values

Tilting toward a target population

The ATO and ATM populations are not subsets of the rows, so weight the average by the tilting function instead

```
1 library(propensity)
2
3 greek_ps <- glm(a ~ 1, data = greek_data, family = binomial()) |>
4   predict(type = "response")
5
6 avg_comparisons(
7   greek_model,
8   variables = "a",
9   wts = ps_tilt(greek_ps, "ato")
10 )
```

Estimate	Std. Error	z	Pr(> z)	S	2.5 %	97.5 %
-1.31e-17	0.24	-5.44e-17	1	-0.0	-0.471	0.471

Term: a

Type: response

Comparison: 1 - 0

Predicting where there are no data

- Step 4 predicts every unit under every level of the exposure
- Where a unit has no counterpart in the other exposure group, that prediction comes from the model rather than from a comparison

Checking extrapolation

```
1 extrapolation <- check_extrapolation(  
2   pos_violations,  
3   exposure,  
4   c(x1, x2, region)  
5 )  
6  
7 extrapolation
```

— Extrapolation

Exposure: "exposure" (binary)

Observations: 1000

Geometric variability: 0.144

Nearby radius (1 x gv): 0.144

Mean fraction nearby: 0.284

Nearest opposite within one geometric variability: 999 of 1000

In opposite-group hull: 784 of 1000

Checking extrapolation

```
1 summary(extrapolation)
```

```
# A tibble: 2 × 5  
  exposure      n mean_gower_min prop_supported prop_in_hull  
  <int> <int>      <dbl>      <dbl>      <dbl>  
1         0   542      0.0269      0.998      0.686  
2         1   458      0.0191      1.000      0.900
```

Nearly a third of the unexposed group falls outside the covariate hull of the exposed group

When extrapolation is a problem

- Weighting shows poor overlap as extreme weights; g-computation absorbs it into the predictions
- The predictions hold if the outcome model gets the functional form right there
- Nothing in the output tells us whether it does

Continuous exposures

Weights or g-computation?

- Continuous exposure weights are unstable under positivity violations
- G-computation avoids the weights, but predicts into the same sparse regions
- We recommend it here, provided we can get the functional form right

Does change in smoking intensity (**smkintensity82_71**) affect weight gain among lighter smokers?

```
1 nhfs_light_smokers <- nhfs_complete |>  
2   filter(smokeintensity <= 25)
```

1. Fit a model for $y \sim x + z$

```
1 nhefs_gcomp_model <- lm(  
2   wt82_71 ~ splines::ns(smkinintensity82_71, df = 3) +  
3     sex + race + age + I(age^2) +  
4     education + smokeintensity + I(smokeintensity^2) +  
5     smokeyrs + I(smokeyrs^2) + exercise + active +  
6     wt71 + I(wt71^2),  
7   data = nhefs_light_smokers  
8 )
```

G-computation still uses a single outcome model, and a continuous exposure lets us fit it flexibly with a natural spline. The remaining steps are identical, except that the levels of step 2 become the exposure values we choose to compare

2 and 3. Set **smkintensity82_71** in each cloned data set

```
1 # set all participants to have no change in intensity
2 unchanged_data <- nhefs_light_smokers |>
3   mutate(smkintensity82_71 = 0)
4
5 # set all participants to smoke 10 fewer cigarettes
6 reduced_data <- nhefs_light_smokers |>
7   mutate(smkintensity82_71 = -10)
```

4. Make predictions using the model on the cloned data sets

```
1 # predict under the data where intensity is unchanged
2 predicted_unchanged <- nhefs_gcomp_model |>
3   augment(newdata = unchanged_data) |>
4   select(unchanged = .fitted)
5
6 # predict under the data where everyone reduces by 10
7 predicted_reduced <- nhefs_gcomp_model |>
8   augment(newdata = reduced_data) |>
9   select(reduced = .fitted)
10
11 nhefs_predictions <- bind_cols(
12   predicted_unchanged,
13   predicted_reduced
14 )
```

5. Calculate the estimate you want

```
1 nhefs_predictions |>
2   summarise(
3     mean_unchanged = mean(unchanged),
4     mean_reduced = mean(reduced),
5     difference = mean_reduced - mean_unchanged
6   )
```

```
# A tibble: 1 × 3
  mean_unchanged mean_reduced difference
  <dbl>         <dbl>         <dbl>
1         1.77         3.21         1.44
```

avg_comparisons() with chosen exposure values

```
1 avg_comparisons(  
2   nhfs_gcomp_model,  
3   variables = list(smkintensity82_71 = c(-10, 0))  
4 )
```

A tibble: 1 × 4

	estimate	std.error	conf.low	conf.high
	<dbl>	<dbl>	<dbl>	<dbl>
1	-1.44	0.39	-2.21	-0.67

`avg_comparisons()` contrasts the higher value against the lower one, so the estimate is the mean at 0 minus the mean at -10. Reversing it gives the answer from step 5: reducing intensity by 10 cigarettes a day increases weight gain by 1.44 kg

Predict across the exposure range

```
1 nhfs_dose_response <- avg_predictions(  
2   nhfs_gcomp_model,  
3   variables = list(smkindensity82_71 = seq(-25, 25, by = 5))  
4 )
```

With a continuous exposure, we don't have to settle for a single contrast: we can ask what the average weight gain would be at every value of the exposure

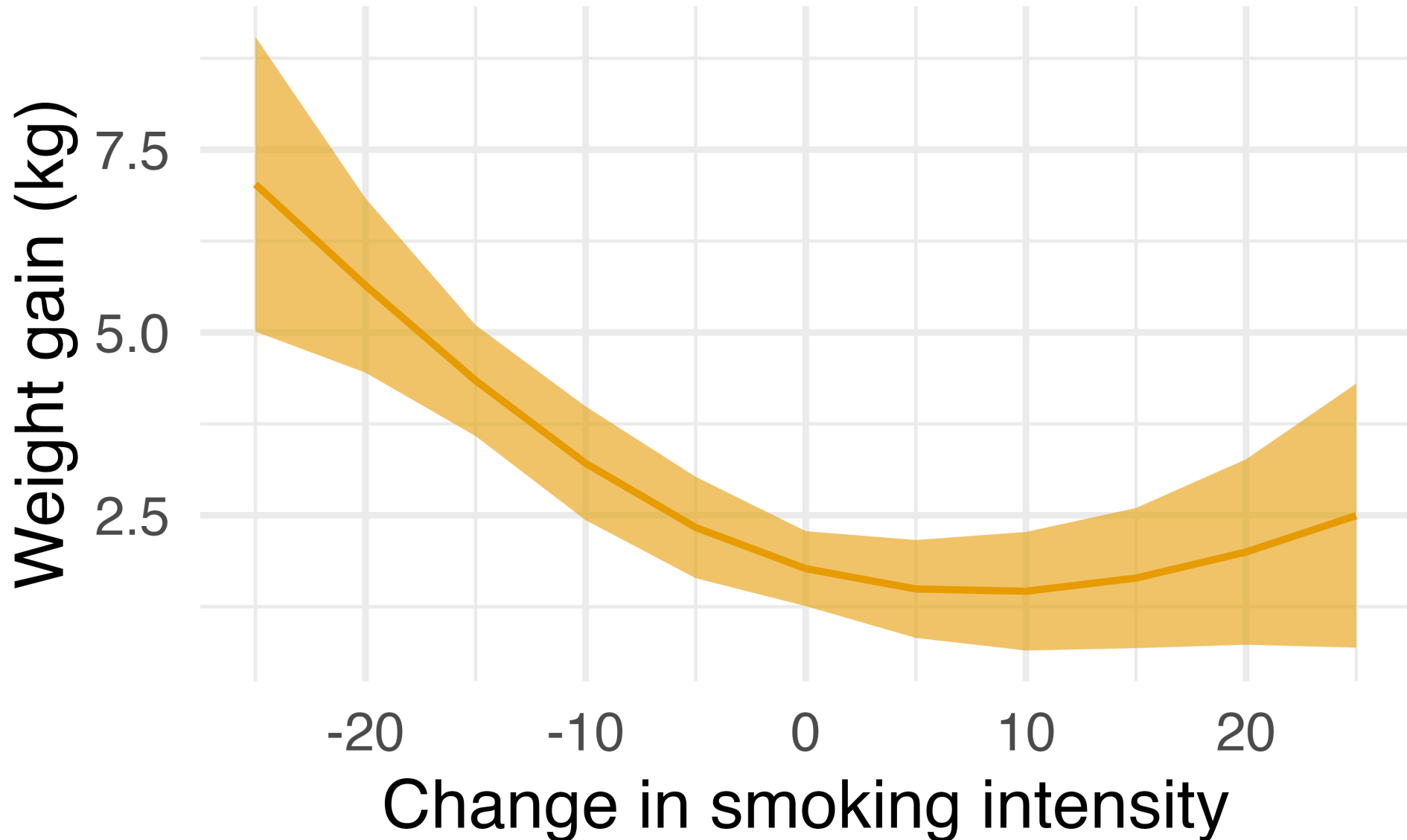
Predict across the exposure range

```
1 nhefs_dose_response
```

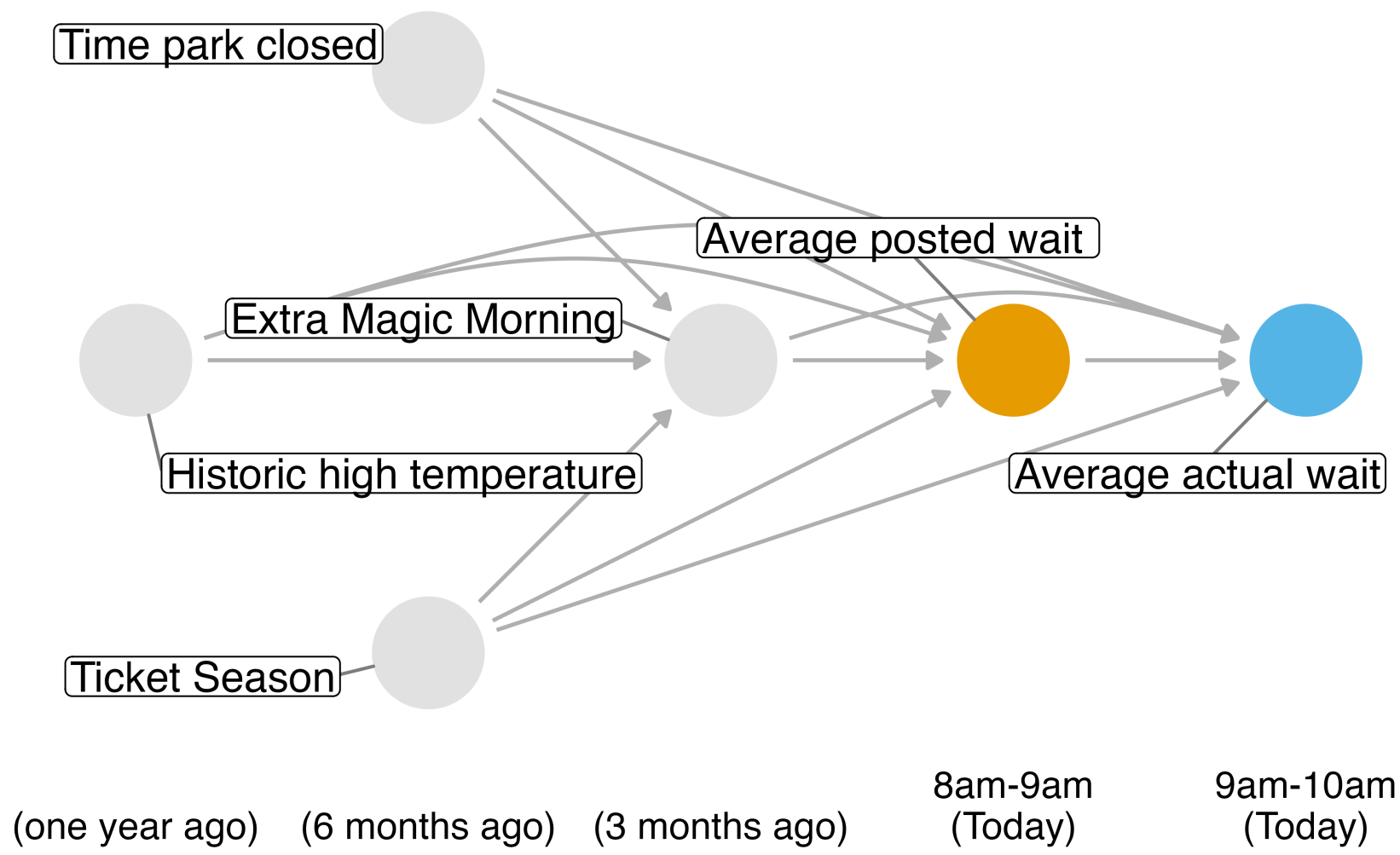
smkintensity82_71	Estimate	Std. Error	z	Pr(> z)	S	2.5 %	97.5 %
-25	7.03	1.028	6.84	< 0.001	36.8	5.013	9.04
-20	5.64	0.608	9.28	< 0.001	65.7	4.451	6.83
-15	4.34	0.387	11.22	< 0.001	94.7	3.584	5.10
-10	3.21	0.397	8.09	< 0.001	50.6	2.433	3.99
-5	2.33	0.353	6.60	< 0.001	34.5	1.640	3.03
0	1.77	0.261	6.77	< 0.001	36.2	1.257	2.28
5	1.49	0.341	4.37	< 0.001	16.3	0.823	2.16
10	1.46	0.414	3.53	< 0.001	11.2	0.650	2.27
15	1.64	0.489	3.36	< 0.001	10.3	0.683	2.60
20	2.00	0.647	3.09	0.00202	8.9	0.729	3.27
25	2.49	0.921	2.71	0.00674	7.2	0.690	4.30

Type: response

The causal dose-response function



Do *posted* wait times at 8 am affect *actual* wait times at 9 am?



Your Turn

Work through Your Turns 1-4 in `12-continuous-g-computation-exercises.qmd`